Métodos de Análisis de Datos 1

Clase 2: Notebooks y Buenas Prácticas

Universidad Nacional del Sur

Contenidos de hoy

¿Qué son los Jupyter Notebooks?

Definición

Documentos **interactivos** que combinan:

- Código ejecutable (Python, R, Julia...)
- Texto narrativo (Markdown, LaTeX)
- Visualizaciones (gráficos, tablas)
- Resultados (outputs)

¿Qué son los Jupyter Notebooks?

Definición

Documentos **interactivos** que combinan:

- Código ejecutable (Python, R, Julia...)
- Texto narrativo (Markdown, LaTeX)
- Visualizaciones (gráficos, tablas)
- Resultados (outputs)

¿Por qué notebooks?

- Facilitan la exploración interactiva
- Documentan el proceso de análisis
- Permiten compartir resultados reproducibles

Estructura de un notebook

Celdas de código

- Ejecutan código (Python, R, etc.)
- Muestran resultados inmediatamente
- Pueden generar gráficos y tablas

Celdas Markdown

- Texto formateado (negritas, cursivas, listas)
- Títulos y secciones
- Ecuaciones matemáticas con LaTeX
- Imágenes y links

Atajos de teclado útiles

Modo comando (Esc)

- `A`: Insertar celda arriba
- `B`: Insertar celda abajo
- `D D`: Borrar celda
- `M`: Convertir a Markdown
- `Y`: Convertir a código

Modo edición (Enter)

- `Shift + Enter`: Ejecutar y avanzar
- `Ctrl + Enter`: Ejecutar sin avanzar
- `Tab`: Autocompletar

Python: Variables y tipos de datos

```python
# Variables y tipos basicos
x = 10                  # int
y = 3.14                # float
nombre = "Ana"          # str
activo = True           # bool

# Listas
numeros = [1, 2, 3, 4, 5]
mezclado = [1, "dos", 3.0, True]

# Diccionarios
persona = {
    "nombre": "Ana",
    "edad": 25,
    "ciudad": "Bahia Blanca"
}
```

Python: Control de flujo

```python
# Condicionales
if x > 5:
    print("Mayor a 5")
elif x == 5:
    print("Igual a 5")
else:
    print("Menor a 5")

# Bucles
for i in range(5):
    print(i)

# List comprehension
cuadrados = [x**2 for x in range(10)]
```

Python: Funciones

```python
# Funcion simple
def saludar(nombre):
    return f"Hola, {nombre}!"

# Funcion con argumentos por defecto
def potencia(base, exponente=2):
    return base ** exponente

# Lambda (funciones anonimas)
cuadrado = lambda x: x**2

# Ejemplo de uso
resultado = potencia(3)       # 9
resultado = potencia(3, 3)    # 27
```

R: Variables y estructuras

```r
# Variables
x <- 10
y <- 3.14
nombre <- "Ana"

# Vectores
numeros <- c(1, 2, 3, 4, 5)
secuencia <- 1:10

# Listas
persona <- list(
    nombre = "Ana",
    edad = 25,
    ciudad = "Bahia Blanca"
)

# Data frames
```

R: Funciones

```r
# Funcion simple
saludar <- function(nombre) {
    paste("Hola,", nombre, "!")
}

# Funcion con argumentos por defecto
potencia <- function(base, exponente = 2) {
    base ^ exponente
}

# Funcion apply (vectorizada)
cuadrados <- sapply(1:10, function(x) x^2)

# Ejemplo de uso
resultado <- potencia(3)       # 9
resultado <- potencia(3, 3)    # 27
```

Buenas prácticas de programación

Código limpio

- Nombres descriptivos de variables
- Comentarios claros y concisos
- Indentación consistente
- Funciones pequeñas y enfocadas

Reproducibilidad

- Documentar dependencias y versiones
- Usar semillas aleatorias (`random.seed()`, `set.seed()`)
- Guardar outputs intermedios
- Registrar parámetros de experimentos

Organización de proyectos

Estructura recomendada

- `data/`: Datos crudos y procesados
- `notebooks/`: Jupyter notebooks
- `src/`: Código fuente (módulos y funciones)
- `outputs/`: Resultados, gráficos, reportes
- `README.md`: Descripción del proyecto
- `requirements.txt` o `environment.yml`

Importante

Nunca subir datos sensibles o credenciales a GitHub

Git: Control de versiones

Comandos básicos

```
# Inicializar repositorio
git init

# Agregar archivos al staging
git add archivo.py
git add .

# Commit (guardar cambios)
git commit -m "Mensaje descriptivo"

# Ver estado
git status

# Ver historial
git log
```

GitHub: Colaboración

Comandos para trabajo remoto

```
# Clonar repositorio
git clone https://github.com/usuario/repo.git

# Subir cambios
git push origin main

# Bajar cambios
git pull origin main

# Crear rama
git branch nueva-funcionalidad
git checkout nueva-funcionalidad
```

Flujo de trabajo típico

R Markdown: Reportes reproducibles

Estructura

```
---
title: "Mi Analisis"
author: "Tu Nombre"
date: "2026-03-16"
output: html_document
---

# Introduccion

Este es texto normal en Markdown.

```{r}
# Este es codigo R
x <- 1:10
mean(x)
```
```

Caso: Dataset Iris

Descripción

- 150 flores de iris
- 3 especies diferentes
- 4 medidas por flor:
  - Largo de sépalo
  - Ancho de sépalo
  - Largo de pétalo
  - Ancho de pétalo

Objetivo

Recorrer todo el ciclo:

1. Cargar datos
2. Explorar
3. Visualizar
4. Modelo simple
5. Interpretar

En el notebook de práctica

Veremos el caso completo implementado en Python y R

Resumen de la clase

- **Jupyter notebooks**: combinan código, texto y visualizaciones
- **Python y R**: sintaxis básica, variables, funciones
- **Buenas prácticas**: código limpio, reproducibilidad, organización
- **Git y GitHub**: control de versiones, colaboración
- **R Markdown**: reportes reproducibles en R

Próxima clase (Unidad 1)

- Fuentes de datos
- Carga de datos en Python y R
- Exploración inicial

Tarea para la próxima clase

1. Completar instalación de herramientas
2. Crear cuenta en GitHub y primer repositorio
3. Configurar Git localmente
4. Hacer el notebook de práctica de U0
5. Subir el notebook al repositorio

Importante

El notebook de práctica está disponible en el campus virtual

¿Preguntas?